Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W0-01 Das Rätsel des Portalwächters · Struktogramm lesen

a) i = 1: 7 > 3 → z = 1 · i = 2: 2 > 7 falsch → z = 1 · i = 3: 9 > 2 → z = 2 · i = 4: 9 > 9 falsch → z = 2 · i = 5: 4 > 9 falsch → z = 2. Rückgabewert: 2.

b) Die Operation zählt, wie oft ein Element größer ist als sein direkter Vorgänger (Anzahl der „Anstiege“).

c) Jede streng aufsteigende Reihung, z. B. [1, 2, 3, 4, 5] → Rückgabewert 4 (= länge(r) − 1). Achtung: [1, 2, 2, 3, 4] liefert nur 3, weil 2 > 2 falsch ist.

W0-02 Vom Code zum Bauplan · Code ↔ Struktogramm

Prüfe dein Struktogramm: Signatur zaehleGerade(r: Reihung von Ganzzahl): Ganzzahl · Zuweisung mit ← (nicht =) · Schleifengrenze länge(r) − 1 · Vergleich sprachunabhängig (mod, =) · „gib anzahl zurück“ als letzter Block nach der Schleife.

Alternative: für jedes x in r · falls x mod 2 = 0.

```
zaehleGerade(r: Reihung von Ganzzahl): Ganzzahl
  anzahl ← 0
  für i von 0 bis länge(r) − 1
    falls r[i] mod 2 = 0
      dann: anzahl ← anzahl + 1
      sonst: –
  gib anzahl zurück
```

W0-03 Spurensuche im Portal · Tracetabelle

b) Rückgabewert: 4. Die Methode liefert den Index des kleinsten Elements (bei mehrfachem Minimum den Index des ersten Vorkommens, weil mit < verglichen wird).

i	r[i]	r[i] < a	a	b
-	-	-	5	0
1	8	falsch	5	0
2	3	wahr	3	2
3	6	falsch	3	2
4	1	wahr	1	4
5	4	falsch	1	4

W0-04 Der kaputte Bauplan · Fehler im Struktogramm

Fehler 1: max ← 0 – enthält r nur negative Werte, wird fälschlich 0 geliefert. Korrektur: max ← r[0].

Fehler 2: Schleife bis länge(r) – im letzten Durchlauf wird auf r[länge(r)] zugegriffen (Index außerhalb der Reihung). Korrektur: bis länge(r) − 1.

Fehler 3: „gib max zurück“ steht in der Verzweigung – die Operation endet schon beim ersten größeren Element; folgt kein größeres Element, gibt es gar keine Rückgabe. Korrektur: „gib max zurück“ nach der Schleife.

```
maximum(r: Reihung von Ganzzahl): Ganzzahl
  // r ist nicht leer
  max ← r[0]
  für i von 1 bis länge(r) − 1
    falls r[i] > max
      dann: max ← r[i]
      sonst: –
  gib max zurück
```

W0-05 Rückwärts durchs Portal · Fehler im Java-Code

Fehler 1: Start bei i = r.length – schon der erste Zugriff r[r.length] liegt außerhalb der Reihung (ArrayIndexOutOfBoundsException). Korrektur: i = r.length - 1.

Fehler 2: Bedingung i > 0 – das Element r[0] wird nie geprüft. Korrektur: i >= 0.

Fehler 3: pos = 0 – kommt x nicht vor, wird 0 geliefert; 0 ist aber ein gültiger Index. Korrektur: -1 zurückgeben.

```
public static int letzterIndexVon(int[] r, int x) {
    for (int i = r.length - 1; i >= 0; i--) {
        if (r[i] == x) {
            return i;
        }
    }
    return -1;
}
```

W0-06 Der Spiegelzauber · Lückencode

a) (1) r.length / 2 · (2) r[i] · (3) r.length - 1 - i · (4) tmp

b) In jedem Durchlauf wird ein Paar (vorne/hinten) getauscht. Liefe i bis r.length - 1, würde jedes Paar ein zweites Mal getauscht – die Reihung stünde wieder in der Ausgangsreihenfolge. Bei ungerader Länge bleibt das mittlere Element stehen (5 / 2 = 2, ganzzahlige Division).

```
public static void umdrehen(int[] r) {
    for (int i = 0; i < r.length / 2; i++) {
        int tmp = r[i];
        r[i] = r[r.length - 1 - i];
        r[r.length - 1 - i] = tmp;
    }
}
```

W0-07 Die Spannweite des Portals · Methode implementieren

Kernpunkte: Leer-Abfrage vor dem Zugriff auf r[0] · min und max mit r[0] initialisieren (nicht mit 0!) · eine Schleife, zwei Vergleiche · Rückgabe max - min.

```
public static int spannweite(int[] r) {
    if (r.length == 0) {
        return 0;
    }
    int min = r[0];
    int max = r[0];
    for (int i = 1; i < r.length; i++) {
        if (r[i] < min) {
            min = r[i];
        }
        if (r[i] > max) {
            max = r[i];
        }
    }
    return max - min;
}
```

W0-08 Das Zählwerk des Portals · Funktionsweise erläutern

a) [2, 7, 8, 12]

b) Es wird eine neue Reihung erg mit derselben Länge wie r erzeugt. summe speichert die laufende Summe aller bisher besuchten Elemente. In jedem Durchlauf wird r[i] zu summe addiert und der aktuelle Stand in erg[i] abgelegt. Damit gilt erg[i] = r[0] + r[1] + … + r[i] (kumulierte Summen). Die neue Reihung wird zurückgegeben.

c) r bleibt unverändert: Aus r wird nur gelesen, geschrieben wird ausschließlich in die neue Reihung erg.

W1-01 Bauplan oder Held? · Klasse vs. Objekt

a) Eine Klasse ist der Bauplan (Typ): Sie legt fest, welche Attribute (z. B. name, leben) und welche Methoden (z. B. nimmSchaden) jedes ihrer Objekte hat. Ein Objekt ist ein konkretes Exemplar dieser Klasse mit eigenen Attributwerten (z. B. name = "Aria", leben = 100). Objekte werden mit new durch Aufruf eines Konstruktors erzeugt.

b) 2 Klassen (Held, Monster), 3 Objekte. Nur new erzeugt Objekte. Held h = aria; kopiert lediglich die Referenz: h und aria verweisen auf dasselbe Objekt.

c) aria: 75 (der Schaden über h trifft dasselbe Objekt) · bruno: 120 · goblin: 30

W1-02 Anatomie eines Monsters · Bestandteile einer Klasse

a) Attribute: name (Zeichenkette), leben (Ganzzahl), staerke (Ganzzahl). Konstruktor: Monster(name, leben, staerke) – heißt wie die Klasse, hat keinen Rückgabetyp und setzt die Anfangswerte der Attribute. Methoden: getLeben(): Ganzzahl und istBesiegt(): Wahrheitswert sind sondierend; nimmSchaden(schaden: Ganzzahl): nichts ist verändernd (leben); angreifen(h: Held): nichts verändert das übergebene Held-Objekt.

b) private: Die Attribute sind nur innerhalb der Klasse Monster zugreifbar. Von außen kann leben also nicht direkt (z. B. auf −50) gesetzt werden – nur über Methoden, die auf gültige Werte achten (Datenkapselung). public: Konstruktor und Methoden sind von außen (z. B. Hauptprogramm, Klasse Held) aufrufbar.

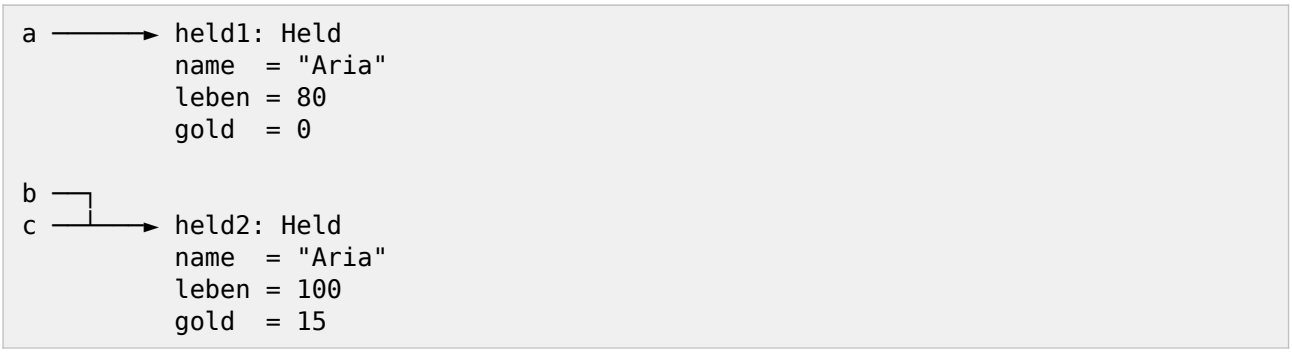
c) Parameter und Attribute heißen gleich; der Parameter verdeckt das Attribut. this.leben bezeichnet das Attribut des aktuellen Objekts, leben (ohne this) den Parameter.

W1-03 Zwei Namen, ein Held · Objektkarten und Referenzen

Es gibt genau zwei Held-Objekte. a und zunächst auch b verweisen auf held1; ab b = c verweisen b und c auf held2.

held1: nimmSchaden(30) → 70, heilen(10) → 80. held2: erhalteGold(15) → 15; erhalteGold(-5) wird ignoriert (nur betrag > 0).

b) a == c → false (zwei verschiedene Objekte; == vergleicht Referenzen) · a.getName().equals(c.getName()) → true (gleicher Inhalt) · a.getLeben() → 80 · c.getGold() → 15



W1-04 Was steckt im Rucksack? · Objektkarten und Referenzen

a) inventar[0] und inventar[2] verweisen auf dasselbe Fackel-Objekt, inventar[1] auf das Trank-Objekt, inventar[3] und inventar[4] sind null. anzahl = 3, waffe = null.

b) Es existieren 2 Gegenstand-Objekte (Fackel, Heiltrank) – die Fackel ist zweimal referenziert, aber nur einmal erzeugt. h.gesamtWert() = 5 + 20 + 5 = 30.

c) true · true · false (g == null) · true

```
h ──► held1: Held
      inventar = [ • , • , • , null, null ]
                   │   │   │
                   │   │   └──► (dasselbe Objekt wie [0])
                   │   └──► g2: Trank
                   │             name = "Heiltrank", wert = 20, heilwert = 30
                   └──► g1: Gegenstand
                              name = "Fackel", wert = 5
      anzahl = 3
      waffe  = null

f ──► g1 (dieselbe Fackel)
```

W1-05 Vom Code zur Klassenkarte · Klassendiagramm

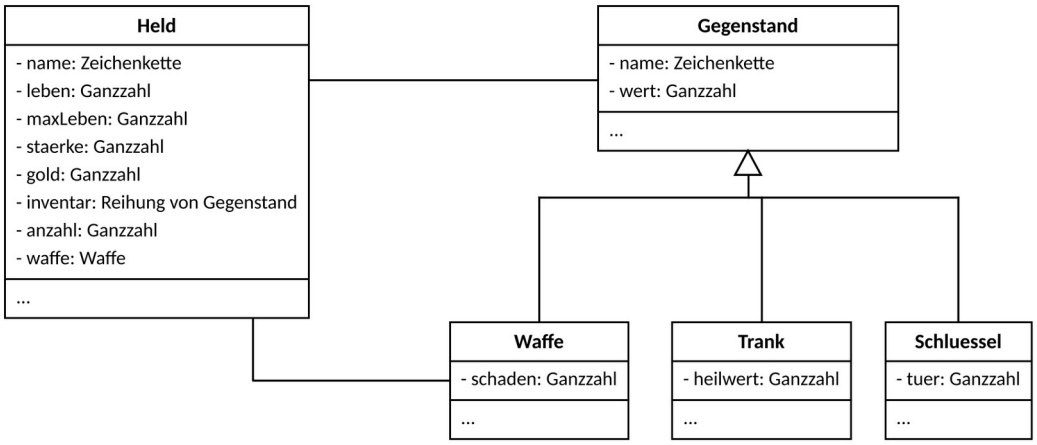
b) Nein. Das Attribut wert ist private, und es gibt keine Methode, die wert verändert (kein Setter). Der Wert wird nur im Konstruktor gesetzt und kann danach nur noch gelesen werden.

```
Gegenstand
──────────────────────────────────────────────
- name: Zeichenkette
- wert: Ganzzahl
──────────────────────────────────────────────
+ c Gegenstand(name: Zeichenkette, wert: Ganzzahl)
+ getName(): Zeichenkette
+ getWert(): Ganzzahl
+ beschreibung(): Zeichenkette
```

W1-06 Der Stammbaum der Gegenstände · Klassendiagramm

a) Vererbung: Waffe, Trank und Schluessel jeweils mit Pfeil mit hohler Dreiecksspitze zu Gegenstand. Assoziationen: einfache, unbeschriftete Linien ohne Kardinalitäten von Held zu Gegenstand (Attribut inventar) und von Held zu Waffe (Attribut waffe). In den Unterklassen stehen nur die zusätzlichen Attribute – name und wert werden geerbt.

b) inventar ist eine Reihung von Gegenstand: Der Held verwaltet beliebige Gegenstände (auch Waffen, Tränke, Schlüssel). waffe ist ein eigenes Attribut vom Typ Waffe: die aktuell ausgerüstete Waffe. Deshalb gibt es eine Linie zu Gegenstand und eine zu Waffe.



W1-07 Die Schmiede der Waffen · Lückencode

a) (1) extends · (2) super(name, wert) – muss die erste Anweisung im Konstruktor sein · (3) super.beschreibung()

b) name und wert sind in Gegenstand private und damit in der Unterklasse Waffe nicht sichtbar → Compilerfehler. Zugriff nur über geerbte öffentliche Methoden: super.beschreibung() oder getName() / getWert().

```
public class Waffe extends Gegenstand {
    private int schaden;

    public Waffe(String name, int wert, int schaden) {
        super(name, wert);
        this.schaden = schaden;
    }

    public int getSchaden() {
        return schaden;
    }

    public String beschreibung() {
        return "Waffe: " + super.beschreibung() + ", Schaden " + schaden;
    }
}
```

W1-08 Ab in den Rucksack · Lückencode

a) (1) null · (2) >= (auch == möglich) · (3) anzahl · (4) anzahl++ bzw. anzahl = anzahl + 1 · (5) true

b) Keine der beiden Teilbedingungen greift auf g zu, beide sind immer auswertbar. Bei g.getWert() muss zuerst g == null geprüft werden: || wertet von links aus und bricht ab, sobald ein Teil wahr ist (Kurzschlussauswertung) – sonst NullPointerException.

```
public boolean aufnehmen(Gegenstand g) {
    if (g == null || anzahl >= inventar.length) {
        return false;
    }
    inventar[anzahl] = g;
    anzahl++;
    return true;
}
```

W1-09 Der falsche Spürsinn · Fehler im Java-Code

Fehler 1: i <= anzahl – inventar[anzahl] ist null (NullPointerException) bzw. bei vollem Inventar Index 5 außerhalb der Reihung. Korrektur: i < anzahl.

Fehler 2: == vergleicht bei Zeichenketten Referenzen, nicht den Inhalt. Korrektur: equals. (Die Online-IDE vergleicht mit == zwar inhaltlich – im Abitur und in echtem Java gilt aber: equals!)

Fehler 3: return false im sonst-Zweig – es wird nur der erste Gegenstand geprüft. Korrektur: return false erst nach der Schleife.

Fehler 4: Nach der Schleife fehlt eine Rückgabe (z. B. bei anzahl == 0 wird die Schleife nicht betreten) → Compilerfehler „missing return statement“.

```
public boolean enthaelt(String name) {
    for (int i = 0; i < anzahl; i++) {
        if (inventar[i].getName().equals(name)) {
            return true;
        }
    }
    return false;
}
```

W1-10 Der gierige Drache · Fehler im Java-Code

Fehler 1: Fehlende Leer-Abfrage – bei anzahl == 0 ist bester == null, bester.getWert() löst eine NullPointerException aus. Korrektur: zuerst if (anzahl == 0) return null;

Fehler 2: Schleife bis inventar.length statt anzahl – bei weniger als 5 Gegenständen wird ein null-Feld erreicht (NullPointerException). Korrektur: i < anzahl.

Fehler 3: >= statt > – bei Gleichstand wird der letzte statt der erste gleich wertvolle Gegenstand geliefert. Korrektur: >. (Start bei i = 1 ist möglich, da inventar[0] schon in bester steht.)

```
public Gegenstand wertvollsterGegenstand() {
    if (anzahl == 0) {
        return null;
    }
    Gegenstand bester = inventar[0];
    for (int i = 1; i < anzahl; i++) {
        if (inventar[i].getWert() > bester.getWert()) {
            bester = inventar[i];
        }
    }
    return bester;
}
```

W1-11 Angriff und Auslese · Methode implementieren

Kernpunkte: a) null-Abfrage vor waffe.getSchaden() · Rückgabe in jedem Pfad. b) Zähler mit 0 initialisieren · Schleife von 0 bis anzahl - 1 (nicht inventar.length – sonst null-Zugriff) · echter Vergleich > (nicht >=) · Rückgabe nach der Schleife.

```
public int angriffswert() {
    if (waffe == null) {
        return staerke;
    }
    return staerke + waffe.getSchaden();
}

public int anzahlTeurerAls(int grenze) {
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (inventar[i].getWert() > grenze) {
            zaehler++;
        }
    }
    return zaehler;
}
```

W1-12 Weg damit! · Methode implementieren

Kernpunkte: Index-Abfrage vor jedem Zugriff (index < 0 || index >= anzahl) · Gegenstand vor dem Verschieben merken · Schleife von index bis anzahl - 2 mit inventar[i] = inventar[i + 1] (Grenze: i + 1 muss < anzahl sein) · letztes belegtes Feld auf null setzen · anzahl verringern · gemerkten Gegenstand zurückgeben.

```
public Gegenstand entfernen(int index) {
    if (index < 0 || index >= anzahl) {
        return null;
    }
    Gegenstand weg = inventar[index];
    for (int i = index; i < anzahl - 1; i++) {
        inventar[i] = inventar[i + 1];
    }
    inventar[anzahl - 1] = null;
    anzahl--;
    return weg;
}
```

W1-13 Die Rüstung des Helden · Verändern und Erweitern

a) Attribut private int ruestung; im Konstruktor ruestung = 0 setzen (in beiden Konstruktoren bzw. nur im zweiten, wenn der erste mit this(…) weiterleitet).

c) Tatsächlicher Schaden 4 − 6 = −2 → wird auf 0 gesetzt, leben bleibt unverändert. Ohne Mindestwert würde der Held durch einen schwachen Treffer um 2 Punkte geheilt – und könnte dabei sogar maxLeben überschreiten.

```
private int ruestung;   // im Konstruktor: ruestung = 0;

public void setRuestung(int r) {
    if (r >= 0) {
        ruestung = r;
    }
}

public void nimmSchaden(int schaden) {
    int echt = schaden - ruestung;
    if (echt < 0) {
        echt = 0;
    }
    leben = leben - echt;
    if (leben < 0) {
        leben = 0;
    }
}
```

W1-14 Die Aufräum-Magie · Funktionsweise erläutern

a) inventar = [Schwert, Heiltrank, null, null, null], anzahl = 2

b) Die erste Schleife durchläuft alle belegten Felder. Jeder Gegenstand mit Wert ≥ 10 wird an die Position j kopiert; j zeigt immer auf das nächste freie Feld für „behaltene“ Gegenstände und zählt sie gleichzeitig. Gegenstände mit Wert < 10 werden übersprungen (überschrieben). Die zweite Schleife setzt die nicht mehr benötigten Felder ab j auf null. Zum Schluss wird anzahl auf j gesetzt. Die Reihenfolge der behaltenen Gegenstände bleibt erhalten.

c) z. B. billigeEntfernen() oder ramschAussortieren()

W1-15 Welche beschreibung() läuft? · Funktionsweise erläutern

b) Welche Methode ausgeführt wird, entscheidet der Typ des Objekts zur Laufzeit, nicht der Typ der Variablen (dynamische Bindung / Polymorphie). Waffe und Trank überschreiben beschreibung() und rufen mit super.beschreibung() die Version der Oberklasse auf. Schluessel überschreibt nicht → es läuft die geerbte Version aus Gegenstand.

c) Der Compiler prüft anhand des deklarierten Typs: x hat den Typ Gegenstand, und Gegenstand besitzt keine Methode getSchaden(). Korrektur: Variable als Waffe deklarieren: Waffe x = new Waffe("Dolch", 15, 5);

```
Waffe: Schwert (50 Gold), Schaden 12
Rostiger Schlüssel (2 Gold)
Trank: Heiltrank (20 Gold), heilt 30
```